



Hassan Premier University
Faculty of Sciences and Techniques of Settat
Department of Computer Science
Programme: Computer Science Engineering
Academic Year 2025–2026



MODULE END-OF-PROJECT REPORT

Centralized AI Monitoring Platform

A Self-Hosted, AI-Powered Infrastructure Supervision Platform

Developed by:

CHAHIDY Rim
SAMDI Wiam
TAÏK Youssef

Supervised by:

Mr. Ezzati

Abstract

Abstract

Infrastructure monitoring is a strategic challenge for any organisation operating servers, from startups to large enterprises. Traditional monitoring tools such as Prometheus, Grafana or Datadog focus on displaying metrics, leaving engineers to interpret data and manually diagnose incidents. This approach generates significant *alert fatigue* and substantially increases the mean time to resolution (MTTR).

This report presents **caimp** (*Centralized AI Monitoring Platform*), a self-hosted open-source platform that repositions infrastructure monitoring around an *answers-first* philosophy: instead of displaying graphs, the dashboard directly answers the question “*Is anything broken, and if so, why?*”.

The architecture is built on nineteen containerised microservices communicating via NATS JetStream. A lightweight agent deployed on each monitored server collects system metrics via the OTLP protocol and automatically reports environment changes (Git deployments, Docker events, package installations, SSH sessions). A change correlation engine links each detected anomaly to modifications that occurred in the thirty minutes preceding the incident, using a temporal proximity score, and submits the analysis to a local LLM (Llama 3.1 8B via Ollama) to produce a natural-language explanation and concrete action recommendations.

The forecasting engine applies Holt-Winters double exponential smoothing over the next 24 hours to anticipate resource saturation. An alert deduplication engine eliminates duplicates within a fifteen-minute window, reducing operational noise. The platform manages dozens of servers in a multi-tenant architecture with complete data isolation per organisation.

Built in one month by a three-student team, the platform totals more than fifty thousand lines of code in Python 3.12, Go 1.22 and TypeScript, and can be installed in minutes via Docker Compose.

Keywords: AIOps, infrastructure monitoring, microservices, large language model, TimescaleDB, NATS JetStream, change correlation, anomaly detection, Holt-Winters, RAG, pgvector.

Contents

Abstract	i
Table of Contents	v
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
1 General Introduction	1
1.1 Context and Motivation	1
1.2 Project Objectives	2
1.3 Working Methodology	2
1.4 Report Structure	2
2 General Context and Project Framework	4
2.1 Modern System Observability	4
2.1.1 The Three Pillars of Observability	4
2.1.2 From Reactive to Predictive Monitoring	5
2.2 The AIOps Domain	5
2.2.1 Definition and Scope	5
2.2.2 Historical Evolution	5
2.3 Problem Statement	5
2.3.1 Alert Fatigue: A Documented Phenomenon	5
2.3.2 The Gap Between Detection and Understanding	6
2.3.3 Problem Formulation	6
2.4 Functional Scope	6
3 State of the Art	7
3.1 Commercial Monitoring Solutions	7
3.1.1 Datadog	7
3.1.2 Grafana + Prometheus	7
3.1.3 Dynatrace	7
3.1.4 Splunk + ITSI	8

3.1.5	Open Source Alternatives	8
3.2	Comparative Table	8
3.3	Anomaly Detection Techniques	8
3.3.1	Static Threshold	8
3.3.2	Z-Score Statistical Detection	8
3.3.3	Rate of Change	9
3.3.4	Holt-Winters Double Exponential Smoothing	9
3.4	Language Models for AIOps	9
3.4.1	Open LLM Landscape	9
3.4.2	Quantisation	9
3.4.3	RAG Pattern	9
3.5	Positioning of CAIMP	10
4	Requirements Analysis and Specification	11
4.1	Actor Identification	11
4.2	Functional Requirements	11
4.3	Non-Functional Requirements	13
4.4	Use Case Diagram	14
4.5	Requirement Traceability Matrix	14
5	Design and Architecture	16
5.1	Architectural Choice: Microservices vs. Monolith	16
5.2	Global Architecture	16
5.3	How the Architecture Works — End-to-End Flows	19
5.3.1	Flow 1 — Metric Collection and Ingestion	19
5.3.2	Flow 2 — AI Explanation and Change Correlation	19
5.3.3	Flow 3 — Resource Forecasting	20
5.3.4	Flow 4 — Real-Time Browser Updates	20
5.3.5	Flow 5 — Dashboard Query	20
5.4	Event Bus: NATS JetStream	20
5.5	Data Model	21
5.5.1	Design Principles	21
5.5.2	Key Tables by Domain	21
5.6	Key Processing Flows	21
5.6.1	Change Correlation Flow	21
5.7	Security: STRIDE Threat Model	22
6	Technology Study	23
6.1	Programming Languages	23
6.2	Backend Frameworks	23
6.3	Database	23

6.4	Artificial Intelligence	24
6.4.1	Holt-Winters — Complete Mathematical Formulation	24
6.5	Infrastructure	24
6.6	Security	25
7	Implementation	26
7.1	Repository Structure	26
7.2	Work Breakdown	26
7.3	Telemetry Writer — Go	26
7.4	Holt-Winters Forecaster — Python/NumPy	27
7.5	Change Correlation Scoring	28
7.6	Answers-First Dashboard — React	28
7.7	Challenges and Solutions	29
8	Testing and Validation	30
8.1	Testing Strategy: The Pyramid	30
8.2	Unit Tests	30
8.3	Integration Tests	31
8.4	Load Tests	31
8.5	Security Tests	32
8.6	Traceability Matrix	32
9	Deployment and Production	33
9.1	Deployment Modes	33
9.2	Complete Installation Procedure	33
9.2.1	Prerequisites	33
9.2.2	Installation steps	33
9.3	Environment Variables	34
9.4	Low-RAM Mode (4 GB)	34
9.5	Platform Self-Monitoring	35
9.6	Generating Demo Metrics	35
9.7	Monitoring a Real Virtual Machine	36
9.8	Accessing the Admin and Monitoring Dashboards	37
10	Results, Discussion and Perspectives	38
10.1	Results Obtained	38
10.1.1	Codebase Metrics	39
10.2	Strengths	39
10.3	Identified Limitations	39
10.4	Future Perspectives	40
11	General Conclusion	41

11.1 Summary of Work Completed	41
11.2 Academic and Professional Contributions	41
11.3 Personal Reflections	42
11.4 Closing Remarks	42
References	43
A Database Schema	45
B API Endpoint Reference	48
B.1 Admin API (port 8001)	48
B.2 Query API (port 8002)	48
B.3 Change Tracker (port 8011)	49
C Docker Compose Extracts	50
D Technical Glossary	52

List of Figures

4.1	Global use case diagram of the CAIMP platform	14
5.1	Global architecture of the CAIMP platform — 7 layers, 19 services	18
8.1	Testing pyramid for CAIMP	30

List of Tables

3.1	Comparison of monitoring solutions across 12 criteria	8
4.1	Traceability matrix: functional requirements — components	14
5.1	Monolith vs. microservices comparison for CAIMP	16
5.2	NATS JetStream streams in CAIMP	21
5.3	STRIDE analysis and countermeasures in CAIMP	22
7.1	Team responsibilities	26
7.2	Key challenges and solutions	29
8.1	Load test results at 1,000 metrics/second	31
8.2	Security tests performed	32
8.3	Functional requirement—test coverage traceability matrix	32
9.1	Deployment modes and resource requirements	33
9.2	Required environment variables	34
9.3	Demo agent environment variable overrides	36
9.4	Web interface access URLs	37
10.1	Objectives vs. results matrix	38
10.2	Quantitative codebase indicators	39

List of Abbreviations

Acronym	Meaning
AI	Artificial Intelligence
AIOps	Artificial Intelligence for IT Operations
API	Application Programming Interface
CA	Certificate Authority
CAIMP	Centralized AI Monitoring Platform
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
CNCF	Cloud Native Computing Foundation
DB	Database
DevOps	Development + Operations
ERD	Entity-Relationship Diagram
FST	Faculty of Sciences and Techniques
GDPR	General Data Protection Regulation
gRPC	Google Remote Procedure Call
HEC	HTTP Event Collector (Splunk)
HNSW	Hierarchical Navigable Small World
HTTP	HyperText Transfer Protocol
IVFFlat	Inverted File Flat (vector index)
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MAE	Mean Absolute Error
MLTK	Machine Learning Toolkit (Splunk)
MTTR	Mean Time To Resolution
mTLS	mutual Transport Layer Security
NATS	Neural Autonomic Transport System
NOC	Network Operations Center
OTLP	OpenTelemetry Protocol

Acronym	Meaning
PKI	Public Key Infrastructure
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RBAC	Role-Based Access Control
RCA	Root Cause Analysis
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software as a Service
SLA	Service Level Agreement
SPL	Search Processing Language (Splunk)
SQL	Structured Query Language
SRE	Site Reliability Engineering
SSH	Secure Shell
SSE	Server-Sent Events
TLS	Transport Layer Security
TSD	Time-Series Database
TS	TypeScript
UHP	Université Hassan Premier
UI	User Interface
UML	Unified Modeling Language
UUID	Universally Unique Identifier
VPS	Virtual Private Server
WS	WebSocket
YAML	YAML Ain't Markup Language

Chapter 1

General Introduction

Introduction

The rapid growth of cloud-native architectures over the past decade has profoundly transformed how organisations deploy and manage their information systems. Modern infrastructures, composed of dozens or even hundreds of servers, containers and distributed services, generate operational data streams of unprecedented density. In this context, system monitoring — once delegated to a network administrator scrutinising a handful of graphs — has become a discipline in its own right, at the heart of organisational resilience.

Yet current monitoring tools, however sophisticated, share a fundamental limitation: they can *detect* that a problem exists, but struggle to explain *why* it appeared. When faced with a “CPU at 98%” alert, the on-call engineer must still conduct a manual investigation lasting anywhere from thirty minutes to several hours. This gap between *detection* and *understanding* is precisely the problem this project sets out to solve.

1.1 Context and Motivation

Industry statistics confirm the scale of the challenge. According to PagerDuty’s *State of Digital Operations* report (2023) [1], the average cost of one hour of downtime exceeds \$300,000 for large enterprises, while the mean time to resolution (MTTR) exceeds 45 minutes in organisations without mature AIOps practices.

For small teams — a startup with two developers managing a VPS, an agency administering ten machines — the situation is even more critical. These teams lack both the human resource of a dedicated DevOps engineer and the budget for commercial solutions costing thousands of euros per month. They need a tool that simply answers: “*Is my server healthy? If not, what should I do?*”

This is the context in which the **caimp** project was born — *Centralized AI Monitoring Platform* — a self-hosted, open-source platform that positions artificial intelligence not as a display gadget, but as the core of the diagnostic process.

1.2 Project Objectives

- O1. Answers-first dashboard:** design an interface whose home page immediately presents the global status, potential issues and recommended actions — requiring no monitoring expertise from the user.
- O2. Automatic change correlation:** develop an engine capable of linking each detected anomaly to recent environment modifications (deployments, restarts, package updates, SSH sessions) to automate root cause investigation.
- O3. Natural-language explanation:** integrate a local language model (without cloud API calls) to produce explanations understandable by a developer with no DevOps training.
- O4. Resource saturation forecasting:** implement a Holt-Winters forecasting engine to anticipate threshold breaches over the next 24 hours.
- O5. Robust microservices architecture:** design a decoupled architecture based on an event bus (NATS JetStream) supporting horizontal scalability and multi-tenant data isolation.
- O6. End-to-end security:** implement mTLS for agents, JWT management, and RLS-based data isolation per organisation.
- O7. Accessible deployment:** make full installation achievable in under ten minutes via Docker Compose, on any machine with 6 GB of RAM.

1.3 Working Methodology

The development of CAIMP followed an agile approach adapted to the academic context. The three-person team held daily synchronisation sessions (*stand-ups*) and organised its work in weekly sprints of five days. The code repository was hosted on GitHub, with a protected main branch and a development branch from which features were developed and tested before integration. Commit messages followed the *Conventional Commits* convention.

1.4 Report Structure

- **Chapter 1:** General context, modern observability, and the AIOps domain.
- **Chapter 2:** State of the art of existing monitoring solutions.
- **Chapter 3:** Functional and non-functional requirements, UML diagrams.
- **Chapter 4:** Platform architecture, data model, and processing flows.
- **Chapter 5:** Technology choices and justifications.
- **Chapter 6:** Implementation details with annotated code extracts.

- **Chapter 7:** Testing strategy and validation results.
- **Chapter 8:** Deployment and production procedures.
- **Chapter 9:** Results, discussion, and perspectives.
- **Chapter 10:** General conclusion.

Conclusion

This project represents the culmination of one month of intensive work, during which a complete infrastructure monitoring platform was designed, developed, tested and documented. The following chapters demonstrate the scientific and technical rigour we sought to maintain throughout this process. The next chapter establishes the contextual foundations essential to understanding the work as a whole.

Chapter 2

General Context and Project Framework

Introduction

This chapter establishes the conceptual framework of the CAIMP project. We describe the technological environment of modern observability and the AIOps domain, then formulate the problem precisely with supporting quantitative data from industry and academic sources.

2.1 Modern System Observability

2.1.1 The Three Pillars of Observability

Observability refers to the ability to infer the internal state of a system from its external outputs [2]. In distributed computing, it traditionally rests on three complementary pillars:

- Metrics** Periodic numerical measurements of system state: CPU usage, memory occupation, request rate, latency. Their collection is lightweight and statistical analysis enables anomaly detection and trend forecasting.
- Logs** Textual records of discrete events: application errors, HTTP requests, service starts. Rich in contextual information, they are indispensable for post-mortem diagnosis but difficult to analyse at scale without appropriate tooling.
- Traces** Representations of a request's journey through the different components of a distributed system. They enable bottleneck localisation and dependency identification between services [3].

The OTLP (*OpenTelemetry Protocol*) [4], standardised by the CNCF, unifies the collection of all three signal types under a common format, facilitating interoperability between instrumentation tools and analysis backends.

2.1.2 From Reactive to Predictive Monitoring

Early generations of monitoring tools, such as Nagios or Zabbix, adopted a purely reactive approach: send an alert when a threshold is exceeded. This philosophy had two major limitations. First, static thresholds generate a significant proportion of false positives and false negatives on variable-load systems. Second, the alert conveys no information about the *cause* of the problem.

The emergence of high-performance time-series databases like TimescaleDB [5] and statistical anomaly detection techniques (Z-score, Isolation Forest [6], Holt-Winters [7]) enabled a shift to a proactive paradigm: detecting drift before it becomes an incident, and anticipating future resource saturation.

2.2 The AIOps Domain

2.2.1 Definition and Scope

AIOps (*Artificial Intelligence for IT Operations*) refers to the application of machine learning and artificial intelligence techniques to IT operations [8]. Gartner, which coined the term in 2017, defines it as “a platform that combines Big Data, machine learning and advanced analytical technologies to directly improve and automate IT operations”.

2.2.2 Historical Evolution

1. **1990s–2000s:** Static threshold monitoring (Nagios, MRTG).
2. **2000s–2010s:** Log and metric aggregation (Splunk, Graphite).
3. **2010s–2018:** Distributed observability (Prometheus, Grafana, Jaeger).
4. **2018–2023:** Emerging AIOps (Dynatrace Davis, Datadog Watchdog).
5. **2023–present:** Generative AIOps (local LLMs, RAG). Explosion of open language models enabling natural-language explanation without cloud dependency.

2.3 Problem Statement

2.3.1 Alert Fatigue: A Documented Phenomenon

Industry studies converge on identifying alert fatigue as one of the most critical operational challenges. According to the PagerDuty 2023 report [1]:

- 73% of engineers report regularly ignoring alerts due to excessive volume;
- 40% of alerts are false positives or duplicates;
- Average MTTR is 4 hours 20 minutes for organisations without AIOps tooling, versus 42 minutes for those that have adopted it.

2.3.2 The Gap Between Detection and Understanding

The standard response to a “CPU at 98%” alert follows an implicit investigation protocol that any operations engineer will recognise: check consumer processes, consult application logs, examine recent deployment history, check slow database queries, consult network metrics, and cross-reference with system events. This sequence, repeated at 2am during on-call duty, is a source of errors and stress, requiring expertise that small teams do not systematically possess.

2.3.3 Problem Formulation

Problem Statement

How can we design a self-hosted infrastructure monitoring platform, accessible to teams without DevOps expertise, capable not only of detecting anomalies but of explaining their cause in natural language and proposing concrete corrective actions, all without dependency on third-party cloud services?

2.4 Functional Scope

In scope: CPU/memory/disk metric collection via OTLP agent; three-algorithm anomaly detection; change correlation root cause analysis; 24-hour Holt-Winters forecasting; answers-first dashboard; alert deduplication; PDF reports; optional Splunk integration; Docker Compose deployment; mTLS and JWT security.

Out of scope: Distributed tracing; automated remediation execution; Windows agent support; mobile interface; federated authentication (SAML, OIDC).

Conclusion

This chapter defined the conceptual framework of the CAIMP project, anchoring the problem in the real challenges of modern monitoring. The analysis of alert fatigue and the gap between detection and understanding fully justifies the AIOps approach adopted. The following chapter surveys existing solutions and positions CAIMP within this ecosystem.

Chapter 3

State of the Art

Introduction

This chapter examines existing monitoring solutions, available anomaly detection techniques, and recent advances in language models applied to IT operations. This comparative analysis is essential to justify the architectural choices of CAIMP and demonstrate the added value of the adopted approach.

3.1 Commercial Monitoring Solutions

3.1.1 Datadog

Datadog is a SaaS monitoring platform founded in 2010, with over 27,000 clients in 2023. Its *Watchdog* AI feature provides automatic anomaly detection, but explanations remain superficial and do not leverage a generative LLM. Pricing: \$15–31/host/month; no on-premise deployment available.

3.1.2 Grafana + Prometheus

The Grafana/Prometheus combination has become the open source observability standard. Prometheus uses a pull-based model with its own TSDB and PromQL query language. Grafana provides visualisation across dozens of data sources. Both are entirely open source, but have **no native AI** and require significant expertise to configure. Alert fatigue is common without intelligent deduplication.

3.1.3 Dynatrace

Dynatrace features the Davis AI engine, a causal analysis system that automatically correlates anomalies across the full application stack. The most advanced root cause analysis on the market, but at \$69/host/month and with no open-source or on-premise option.

3.1.4 Splunk + ITSI

Splunk Enterprise with ITSI [9] adds MLTK-based anomaly detection and event correlation. CAIMP provides optional integration with Splunk as a secondary metric source. Already-equipped organisations can leverage CAIMP’s explainability layer on top of Splunk’s detection.

3.1.5 Open Source Alternatives

SigNoz (built on ClickHouse), Netdata (real-time single-host), Zabbix (mature but steep learning curve), and Nagios (static threshold pioneer, now outdated) all lack native AI explanation capabilities.

3.2 Comparative Table

Table 3.1: Comparison of monitoring solutions across 12 criteria

Criterion	Datadog	Grafana/Prometheus	PyMatrace	SigNoz	Zabbix	caimp
Price/host/mo	\$15–31	Free	\$69	Free	Free	Free
On-premise	No	Yes	No	Yes	Yes	Yes
Native AI	Partial	No	Advanced	No	No	Yes
LLM explanation	No	No	No	No	No	Yes
RAG + runbooks	No	No	No	No	No	Yes
Change corr.	No	No	Partial	No	No	Yes
24h forecast	Yes	No	Yes	No	No	Yes
Alert dedup.	Yes	Via AM	Yes	Partial	No	Yes
Multi-tenant	Yes	Limited	Yes	Yes	Yes	Yes
Deploy <10m	No	No	No	No	No	Yes
Answers-first	No	No	Partial	No	No	Yes

3.3 Anomaly Detection Techniques

3.3.1 Static Threshold

The simplest method: alert when a value exceeds a predefined threshold. Fast and deterministic, but prone to false positives on variable-load systems. Implemented in CAIMP as a first rapid filter.

3.3.2 Z-Score Statistical Detection

The Z-score measures how many standard deviations a reading is from the historical mean (Equation 3.1):

$$z = \frac{x - \mu}{\sigma} \quad (3.1)$$

A value $|z| > 3$ is considered anomalous. This method adapts automatically to the system’s normal behaviour.

3.3.3 Rate of Change

The discrete derivative of the time series:

$$\Delta_t = \frac{x_t - x_{t-1}}{x_{t-1}} \times 100\% \quad (3.2)$$

Particularly effective for detecting rapid saturations that Z-score on a long window might miss.

3.3.4 Holt-Winters Double Exponential Smoothing

[7, 10]:

$$L_t = \alpha x_t + (1 - \alpha)(L_{t-1} + T_{t-1}) \quad (3.3)$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \quad (3.4)$$

$$\hat{x}_{t+h} = L_t + h \cdot T_t \quad (3.5)$$

Parameters used in CAIMP: $\alpha = 0.3$, $\beta = 0.1$. Confidence intervals: $\pm 1.28\sigma$ residuals ($\approx 80\%$ coverage).

3.4 Language Models for AIOps

3.4.1 Open LLM Landscape

Llama 3.1 8B [11] was selected for CAIMP for its excellent quality/performance ratio in Q4_K_M quantised format: 4.7GB, inference latency under 2 seconds on standard hardware. Alternatives evaluated: Mistral 7B (slightly larger context window), Phi-3 Mini (excellent on constrained hardware), Gemma 7B (less well-aligned for operational tasks).

3.4.2 Quantisation

Q4_K_M quantisation [12] reduces Llama 3.1 8B from 16GB (fp16) to 4.7GB with less than 2% quality degradation on reasoning benchmarks, making local LLM inference feasible on standard server hardware.

3.4.3 RAG Pattern

The Retrieval-Augmented Generation pattern [13]:

1. **Indexing:** documents (runbooks, past incidents) are embedded via `nomic-embed-text` and stored in pgvector [14].
2. **Retrieval:** at incident time, a query embedding is computed and the k most similar documents are retrieved via HNSW cosine similarity search.
3. **Augmentation:** retrieved documents are injected into the LLM prompt as context before generation.

3.5 Positioning of CAIMP

The state of the art reveals a gap between expensive commercial AIOps solutions (Datadog, Dynatrace) and open source tools without AI (Grafana/Prometheus). CAIMP occupies this space with four key differentiators: natural-language explanation via a local LLM; automatic change correlation (absent from all compared open source solutions); a lightweight modern event-driven architecture; and ten-minute deployment on any machine.

Conclusion

The state of the art shows that while monitoring tools are plentiful, none simultaneously combines self-hosting, accessibility for non-experts, and natural language explanation via a local LLM. This gap fully justifies the creation of CAIMP. The following chapter translates these findings into concrete functional and non-functional specifications.

Chapter 4

Requirements Analysis and Specification

Introduction

This chapter formalises the system requirements as functional and non-functional needs, identifies the actors, and presents the UML models that guide the design phase. The analysis drew on the comparative study from the previous chapter, supplemented by concrete usage scenarios.

4.1 Actor Identification

Administrator

Most privileged role. Manages organisations, users, servers and platform configuration. Accesses the Admin API (port 8001).

Operator Technical user who consults dashboards, manages runbooks, configures alert rules and interprets AI analyses. Role `devops` in the database.

Viewer Read-only access. Consults dashboards and PDF reports. Designed for non-technical stakeholders.

System Agent

Non-human actor. The `caimp-agent` daemon process runs on monitored servers, authenticates via mTLS certificate and sends metrics via OTLP.

4.2 Functional Requirements

FR1 The agent must collect CPU, memory and disk metrics every 15 seconds and transmit them via OTLP/HTTP.

FR2 The agent must self-register with the platform using a single-use enrollment token.

- FR3** The agent must use an mTLS certificate issued by the internal CA for every connection.
- FR4** The agent must detect and report Docker events automatically.
- FR5** The agent must monitor package installation logs and SSH events from authentication logs.
- FR6** The platform must detect anomalies in real time using at least three distinct algorithms.
- FR7** Detected anomalies must be published on NATS JetStream for asynchronous processing.
- FR8** The AI analysis engine must query the vector database to retrieve similar past incidents and relevant runbooks.
- FR9** The AI analysis engine must submit a structured prompt to Ollama and validate the JSON response.
- FR10** The AI explanation must contain: natural-language explanation, probable root cause, recommended action, and confidence level.
- FR11** The correlation engine must query `change_events` within the 30-minute window before each anomaly.
- FR12** The correlation engine must score each change by temporal proximity and type, and produce a ranked list of probable causes.
- FR13** The forecast engine must compute Holt-Winters predictions over 24 hours for each server/metric pair.
- FR14** The forecast engine must calculate the *time to critical* — the predicted delay before threshold breach.
- FR15** Alert deduplication must suppress identical alerts within 15 minutes.
- FR16** The query API must expose a `/dashboard/summary` endpoint aggregating global status, potential issues and recommended actions.
- FR17** The user interface must present the global status, potential issues sorted by criticality and numbered recommended actions on the home page.
- FR18** The platform must allow runbook management (create, edit, delete) with automatic embedding re-indexing.
- FR19** The report module must generate a multi-section PDF report readable by non-technical stakeholders.

- FR20** The interface must provide an SSE chatbot for natural-language questions about incidents and servers.
- FR21** The platform must receive GitHub and GitLab webhooks for deployment events.
- FR22** The administrator must be able to configure notification channels (Slack, PagerDuty).
- FR23** Status updates must be pushed in real time to connected browsers via Web-Socket.
- FR24** The platform must support multi-tenant isolation: one organisation's data must never be accessible by another.
- FR25** Each service must provide a health endpoint used by Docker healthchecks.

4.3 Non-Functional Requirements

Performance: OTLP batch processing <50 ms latency p50; throughput $\geq 1,000$ metrics/second; API GET responses <300 ms for 50 servers; full AI explanation <30 seconds.

Security: Agent mTLS (TLS 1.3); single-use enrollment tokens expiring in 30 minutes; JWT HS256 with 15-minute expiry; RLS PostgreSQL; AES-256-GCM encrypted Splunk secrets; immutable audit log.

Availability: 99.5% uptime target; Docker healthchecks with **unless-stopped** restart; local SQLite buffering in agent; JetStream message durability on disk.

Maintainability: Independent, individually deployable services; versioned DB migrations; conventional commits; inline documentation.

4.4 Use Case Diagram

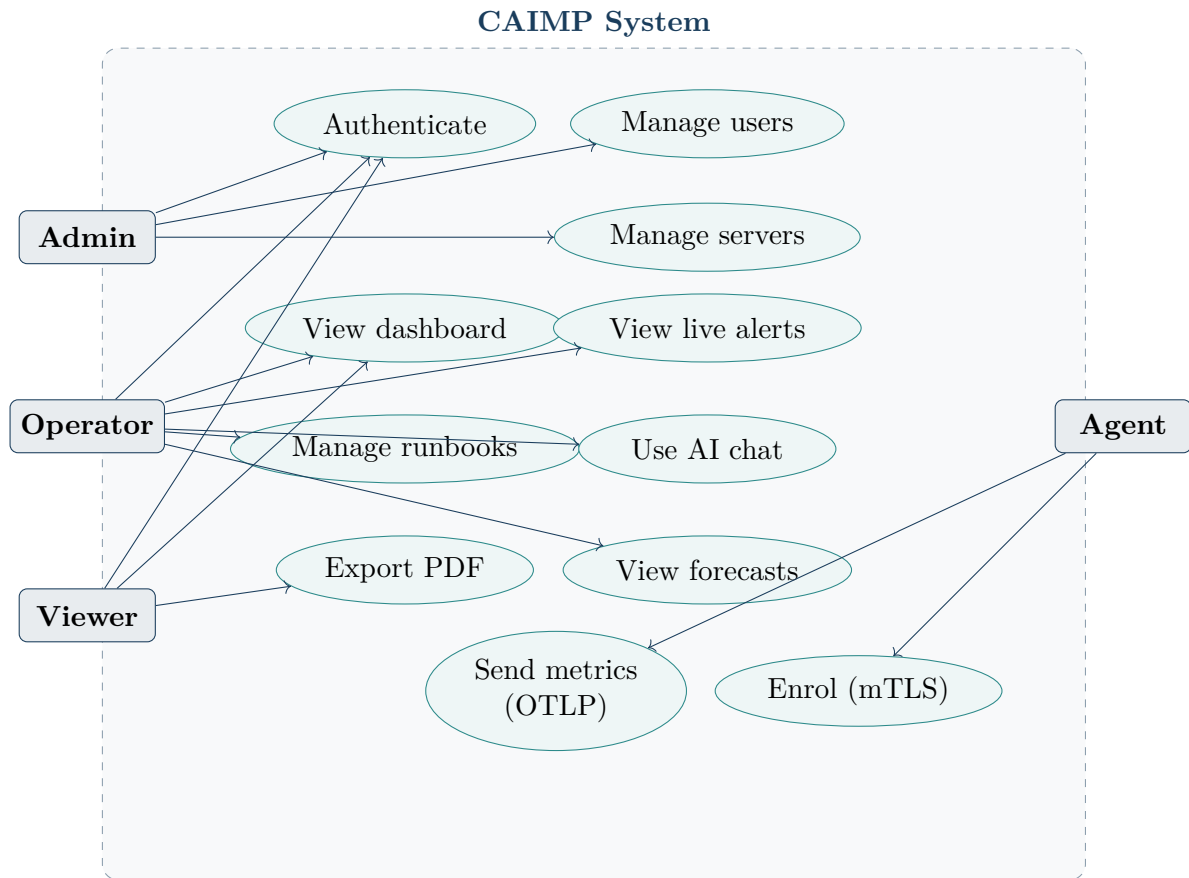


Figure 4.1: Global use case diagram of the CAIMP platform

4.5 Requirement Traceability Matrix

Table 4.1: Traceability matrix: functional requirements — components

FR	Requirement	Component	Module
FR1	OTLP metric collection	caimp-agent	collectors.py
FR6	3-algorithm anomaly detection	telemetry-writer	detector.go
FR8–FR10	AI explanation	ai-worker	prompt_builder.py
FR11–FR12	Change correlation	correlation-engine	main.py
FR13–FR14	Holt-Winters forecast	forecast-engine	forecaster.py
FR15	Alert deduplication	alert-engine	worker.py
FR16–FR17	Answers-first dashboard	query-api	dashboard.py
FR18	Runbook CRUD + RAG	query-api	incidents.py
FR21	GitHub/GitLab webhooks	change-tracker	main.py
FR23	WebSocket push	ws-gateway	main.py
FR24	Multi-tenant isolation	PostgreSQL	RLS migrations

Conclusion

The requirements analysis identified 25 functional requirements covering the full scope of CAIMP, and formalised non-functional requirements around performance, security, availability and maintainability. The UML models presented constitute the contract between analysis and the design covered in the next chapter.

Chapter 5

Design and Architecture

Introduction

This chapter is the technical heart of the report. It describes the CAIMP microservices architecture, data model, main processing flows and design decisions that guided development. Every architectural choice is justified against the alternatives evaluated.

5.1 Architectural Choice: Microservices vs. Monolith

The microservices architecture was selected for three decisive reasons: (1) Holt-Winters forecasting and LLM inference are compute-intensive tasks that must not block the query APIs; (2) the Go telemetry writer benefits from goroutines for concurrent ingestion without Python GIL limitations; (3) future metric sources (Kubernetes, AWS CloudWatch) can be added as independent services without modifying existing ones.

Table 5.1: Monolith vs. microservices comparison for CAIMP

Criterion	Modular Monolith	Microservices
Deployment	Simple (one process)	Complex (Docker Compose)
Scalability	Vertical only	Horizontal per service
Fault isolation	One crash affects all	Graceful degradation
Mixed languages	Impossible	Go + Python
AI decoupling	LLM in API process	Dedicated worker

5.2 Global Architecture

The platform comprises 19 containerised services organised into four functional layers:

Ingestion layer Receives metrics from agents (`telemetry-writer`, `splunk-hec-bridge`) and changes (`change-tracker`).

Event layer NATS JetStream bus providing asynchronous communication.

Processing layer Consumer services processing events independently.

Presentation layer REST APIs, WebSocket Gateway and React frontend.

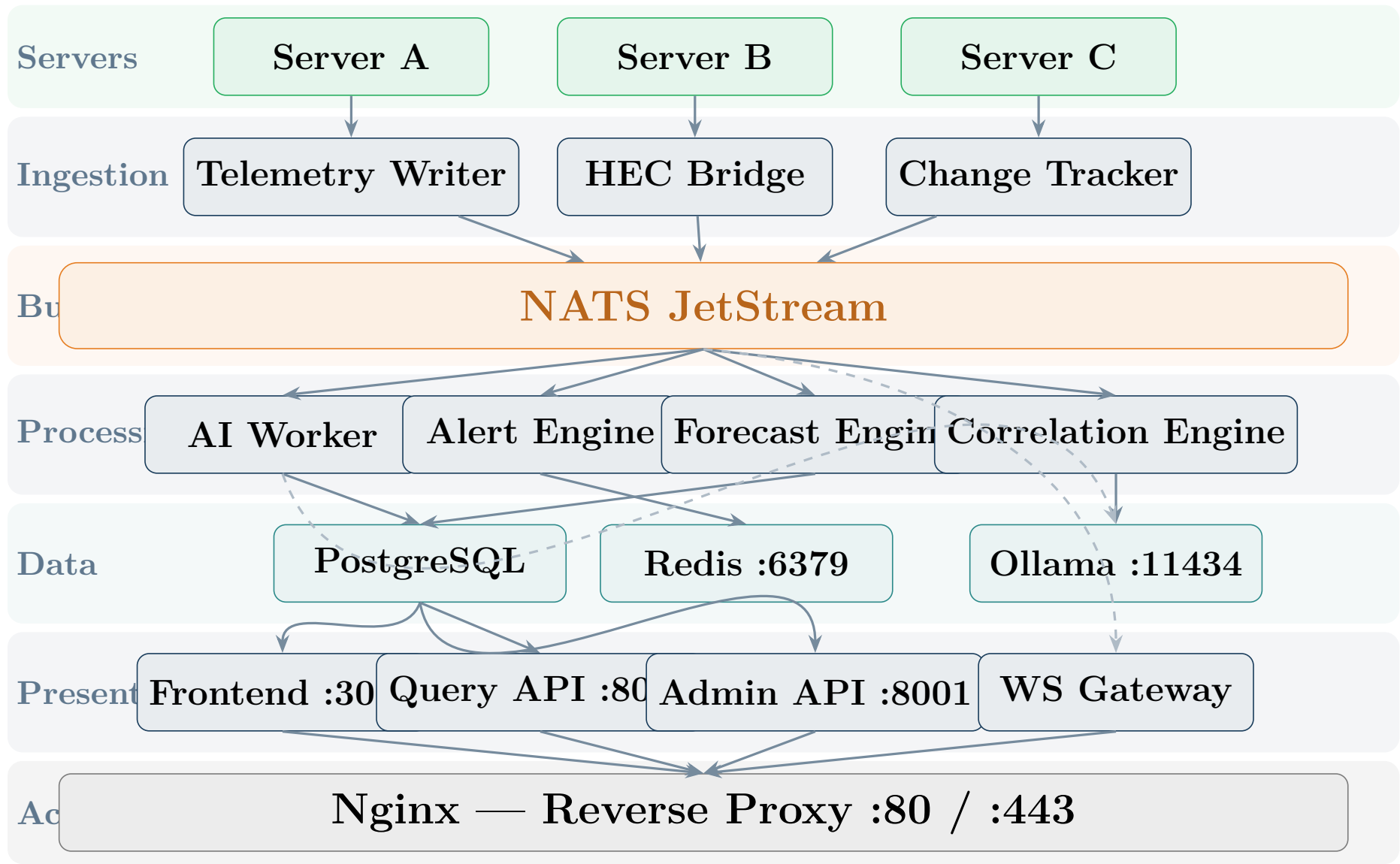


Figure 5.1: Global architecture of the CAIMP platform — 7 layers, 19 services

5.3 How the Architecture Works — End-to-End Flows

Figure 5.1 shows the static structure of the platform. This section describes its dynamic behaviour: how data travels through each layer, from collection on the monitored server to a natural-language explanation appearing in the operator’s browser.

5.3.1 Flow 1 — Metric Collection and Ingestion

The CAIMP agent is a Python daemon deployed on each target server. Every 15 seconds it reads system indicators (CPU, memory, disk) via `psutil`, encodes them as OTLP/HTTP (protobuf), and sends them to the **Telemetry Writer** (port 4318). This Go service:

1. **Receives** the OTLP batch and extracts metric data points;
2. **Persists** the values to the `metrics` hypertable (TimescaleDB) via a bulk `COPY` insert;
3. **Detects** anomalies in parallel using three algorithms (static threshold, Z-score, rate of change);
4. **Publishes** each anomaly on the NATS subject `anomaly.detected.{org_id}.{severity}`.

In parallel, the agent runs four *change reporters* that continuously watch Docker events, package installations, SSH sessions and cron jobs. These events are sent to the **Change Tracker** (port 8011), which stores them in `change_events` and publishes them on `change.detected.>`.

5.3.2 Flow 2 — AI Explanation and Change Correlation

As soon as an `anomaly.detected.*` message arrives on the NATS bus, **two independent consumer services** wake up:

AI Worker For each anomaly it:

1. Queries PostgreSQL for the last 30 minutes of metric history;
2. Queries pgvector by cosine similarity to retrieve the most similar runbooks and past incidents (*Retrieval-Augmented Generation*, *RAG*);
3. Builds a structured prompt targeting non-DevOps developers and sends it to Ollama (Llama 3.1 8B LLM);
4. Validates the JSON response, inserts it into `ai_explanations`, then publishes `ai.explanation.read` on NATS.

Correlation Engine In parallel it performs causal analysis:

1. Queries `change_events` in the window $[t - 30\text{min}, t]$ for the same server;
2. Scores each candidate change: $s = e^{-0.1 \cdot \Delta t} \times w_{\text{type}}$ (temporal decay $\times$ change-type weight);

3. Sends the top 3 candidates to Ollama for a causal narrative;
4. Persists the result in `root_cause_analyses` and publishes `rootcause.ready.*` on NATS.

5.3.3 Flow 3 — Resource Forecasting

The `Forecast Engine` runs autonomously on two periodic loops:

Forecast loop (every 30 min) For each server/metric pair, fetches the last 6 hours of data, applies Holt-Winters double exponential smoothing, generates 24 hourly forecast points with $\pm 1.28\sigma$ confidence intervals, computes *time-to-critical*, requests an LLM narrative from Ollama, and persists everything in `server_forecasts`.

Evaluation loop (every 60 min) Compares past forecasts against actual values, stores the mean absolute error (MAE), and feeds historical accuracy into the next LLM prompt to calibrate expressed confidence.

5.3.4 Flow 4 — Real-Time Browser Updates

The `WS Gateway` maintains persistent WebSocket connections with all connected browsers. It subscribes to: `anomaly.detected.>`, `ai.explanation.ready.>`, `rootcause.ready.>`, and `forecast.critical.>`.

When a message arrives, the gateway fan-outs it to all WebSocket clients of the relevant organisation (multi-tenant isolation enforced). The dashboard refreshes with no polling — detection-to-display latency is under 3 seconds.

5.3.5 Flow 5 — Dashboard Query

When an operator opens the dashboard, the browser sends `GET /api/v1/dashboard/summary` to the `Query API` (via Nginx). It aggregates server statuses, 24-hour anomalies, at-risk forecasts, and the latest root-cause analyses into a structured response: *overall_status*, *potential_issues* (sorted by criticality), and *recommended_actions* (numbered). The answers-first interface presents this immediately without requiring the operator to read graphs.

5.4 Event Bus: NATS JetStream

NATS JetStream [15] was selected over Kafka and RabbitMQ for its 30 MB memory footprint, sub-millisecond latency, and simple configuration while offering the durability guarantees needed (disk persistence, acknowledgements).

Table 5.2: NATS JetStream streams in CAIMP

Stream	Subjects	Description	Retention	Consumers
METRICS	metric.received.>	Raw metric batches	24h	2
ANOMALIES	anomaly.detected.>	Detected anomalies	7d	3
AI_OUT	ai.explanation.ready.>	AI explanations	30d	1
CHAT	chat.requested.>	Chat requests	1h	1
AUDIT	audit.log.>	Audit events	90d	0
FORECASTS	forecast.>	Forecast results	30d	1
CHANGES	change.detected.>	System changes	7d	2
ROOT_CAUSE	rootcause.ready.>	Root cause analyses	30d	1

5.5 Data Model

5.5.1 Design Principles

Two fundamental principles guided the data model:

1. **Multi-tenant isolation via RLS:** every table carrying application data contains an `org_id` column. RLS policies guarantee a query can only access the current organisation's rows (via `current_org_id()`).
2. **Hypertables for time-series data:** `metrics` and `change_events` are TimescaleDB hypertables partitioned automatically by time chunks. TimescaleDB applies columnar compression after 7 days, reducing storage by a factor of 10–20.

5.5.2 Key Tables by Domain

Authentication & Management: `organizations`, `users`, `servers`, `ca_config`, `agent_certs`, `enrollment_tokens`, `refresh_token_blacklist`, `audit_log`.

Metrics & Anomalies: `metrics` (hypertable), `anomaly_events`, `alert_rules`, `notifications`.

AI: `ai_explanations`, `server_forecasts`, `rag_documents`, `runbooks`.

Change Correlation: `change_events` (hypertable), `root_cause_analyses`.

Splunk: `splunk_incidents`, `splunk_ai_explanations`, `ai_feedback`, `org_config`.

5.6 Key Processing Flows

5.6.1 Change Correlation Flow

1. Correlation Engine receives `anomaly.detected.*` message;
2. Queries `change_events` in window $[t - 30\text{min}, t]$ for the same server;
3. Scores each change: $s = e^{-0.1 \cdot \Delta t} \times w_{\text{type}}$, where Δt is minutes before anomaly and $w_{\text{type}} \in [0.3, 1.0]$;

4. Top 3 changes sent to Ollama with anomaly context;
5. Result persisted in `root_cause_analyses`, published to `rootcause.ready.{org_id}`.

5.7 Security: STRIDE Threat Model

Table 5.3: STRIDE analysis and countermeasures in CAIMP

Threat	Example	Countermeasure
Spoofing	Malicious agent sending forged metrics	mTLS ECDSA P-256; JWT
Tampering	Metric modification in transit	TLS 1.3; AEAD integrity
Repudiation	Deny an administrative action	Immutable <code>audit_log</code>
Info Disclosure	Access to another org's data	RLS PostgreSQL; Docker bridge
DoS	Metric flood from compromised agent	Nginx rate limiting
EoP	Viewer accessing admin routes	RBAC role check per endpoint

Conclusion

This chapter presented the architectural foundations of CAIMP: the microservices design, four-layer organisation, NATS JetStream event bus, relational data model and key processing flows. This architecture ensures the separation of concerns necessary for platform scalability and maintainability. The next chapter justifies the technology choices that give this architecture substance.

Chapter 6

Technology Study

Introduction

Every technology integrated into CAIMP was evaluated systematically: functional relevance, maturity, community support, memory footprint and coherence with the overall system. This chapter presents and justifies each choice.

6.1 Programming Languages

Python 3.12 is used for all API services and background workers. Version 3.12 brings 10–15% CPython performance improvements and enhanced error messages. The **async/await** pattern is used pervasively for I/O-bound services awaiting both NATS messages and Ollama responses.

Go 1.22 was chosen for the Telemetry Writer for its native concurrency model (goroutines + channels), ideal for parallel metric ingestion. The Go compiler produces a static binary, simplifying Docker packaging. A Go server starts in milliseconds and consumes less than 20 MB RAM at rest.

TypeScript (React 18) powers the frontend. Static typing prevents interface mismatch bugs across the numerous frontend-backend contracts in CAIMP.

6.2 Backend Frameworks

FastAPI [16] was selected over Flask (synchronous by default) and Django REST Framework (too heavy for microservices) for its automatic OpenAPI/Swagger documentation generation, Pydantic payload validation, native async/await support, and performance comparable to Node.js.

asyncpg provides a native C-based asyncio PostgreSQL driver that is 2–3x faster than psycopg2 via binary protocol and zero-copy read paths.

6.3 Database

PostgreSQL 16 + TimescaleDB [5, 17]:

- **Hypertables:** automatic time-based partitioning for `metrics` and `change_events` (7-day chunk intervals). Queries on time ranges read only relevant chunks (*chunk exclusion*).
- **Columnar compression:** after 7 days, chunks are compressed in columnar format. Average compression ratio for numeric metrics: 12:1.
- **Retention policies:** automatic cleanup of old data chunks.

pgvector [14] adds `VECTOR(768)` columns and the HNSW index for approximate nearest-neighbour search with $O(\log n)$ complexity and sub-millisecond query latency.

Redis 7.2 is used for: alert deduplication (NX keys, 15-min TTL); API cache (30 seconds); live metric values from HEC Bridge (90-second TTL); JWT refresh token blacklist.

6.4 Artificial Intelligence

Ollama serves two models locally:

- `llama3.1:8b-instruct-q4_K_M`: Q4_K_M quantisation reduces from 16 GB (fp16) to 4.7 GB with <2% quality degradation [12];
- `nomic-embed-text` [18]: 270 MB embedding model producing 768-dimensional vectors optimised for semantic search.

6.4.1 Holt-Winters — Complete Mathematical Formulation

$$L_0 = x_1 \quad (\text{level initialisation})$$

$$T_0 = \frac{x_n - x_1}{n - 1} \quad (\text{trend initialisation})$$

$$L_t = \alpha \cdot x_t + (1 - \alpha)(L_{t-1} + T_{t-1}) \quad (\text{level update})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \quad (\text{trend update})$$

$$\hat{x}_{t+h} = L_t + h \cdot T_t \quad (\text{forecast at } h \text{ steps ahead})$$

Confidence intervals at $\pm 1.28\sigma$ ($\approx 80\%$ coverage): $\hat{x}_{t+h} \pm 1.28 \cdot \hat{\sigma}_r$, where $\hat{\sigma}_r = \sqrt{\frac{1}{n} \sum r_t^2}$.
Parameters: $\alpha = 0.3$, $\beta = 0.1$.

6.5 Infrastructure

Docker + Docker Compose v2 [19]: 19 services with healthchecks, dependency conditions, named volumes, and a bridge network (`caimp_net`) isolating all services from external access except via Nginx.

NATS JetStream [15]: selected over Kafka (512 MB + ZooKeeper) and RabbitMQ for its 30 MB footprint, sub-ms latency, and simple configuration.

OTLP [4]: CNCF standard for metrics/traces/logs transmission. Ensures future compatibility with other observability backends.

6.6 Security

JWT [20]: HS256 algorithm, 64-byte secret, 15-minute access token expiry, 7-day refresh token, Redis blacklist.

mTLS [21]: ECDSA P-256 certificates issued by internal CA stored AES-256-GCM encrypted in PostgreSQL. Single-use enrollment tokens expire after 30 minutes. Revocation checked on every connection.

Conclusion

The combination of Python/Go for the backend, TimescaleDB for time-series data, NATS for the event bus, and Ollama for local LLM inference constitutes a coherent, performant and accessible stack. The next chapter details the concrete implementation of these choices.

Chapter 7

Implementation

Introduction

This chapter presents the implementation details of CAIMP’s key modules, illustrated with annotated code extracts. We describe the difficulties encountered, the solutions adopted, and the division of work within the team.

7.1 Repository Structure

The CAIMP v2 repository uses a monorepo structure, with each service in its own directory under `services/`. The 14 database migrations are versioned under `infra/postgres/migrations/`, and the shared Python NATS client library is in `libs/nats_client/`.

7.2 Work Breakdown

Table 7.1: Team responsibilities

Member	Primary modules
CHAHIDY Rim	Telemetry Writer (Go), Alert Engine, React frontend (Dashboard, Forecasts, Incidents), CSS design system
SAMDI Wiam	AI Worker, AI Orchestrator, Correlation Engine, RAG pipeline, LLM prompts, PDF reports
TAÏK Youssef	Admin API, Query API, database (migrations, RLS), Change Tracker, CAIMP agent, DevOps (Docker Compose, Makefile, scripts)

7.3 Telemetry Writer — Go

```
1 // zScore computes Z-score over the last 100 historical values.
2 func (d *Detector) zScore(ctx context.Context,
3     points []receiver.MetricPoint) []AnomalyEvent {
4
5     var events []AnomalyEvent
6     for _, p := range points {
```

```

7         rows, _ := d.pool.Query(ctx,
8             'SELECT value FROM metrics WHERE server_id=$1
9             AND metric_name=$2 ORDER BY time DESC LIMIT 100',
10            p.ServerID, p.MetricName)
11     var vals []float64
12     for rows.Next() {
13         var v float64
14         if err := rows.Scan(&v); err == nil {
15             vals = append(vals, v)
16         }
17     }
18     rows.Close()
19     if len(vals) < 30 { continue } // insufficient history
20
21     mean, std := meanStd(vals)
22     if std < 1e-9 { continue }
23     z := math.Abs((p.Value - mean) / std)
24     if z >= d.cfg.ZScoreThreshold {
25         sev := "warning"
26         if z >= d.cfg.ZScoreThreshold*1.5 { sev = "critical" }
27         events = append(events, AnomalyEvent{
28             Detector: "zscore", Severity: sev,
29             Value: p.Value, MetricName: p.MetricName,
30         })
31     }
32 }
33 return events
34 }

```

Listing 7.1: Z-score detector in Go

7.4 Holt-Winters Forecaster — Python/NumPy

```

1 def _holt_linear(values: np.ndarray, horizon: int,
2                 alpha: float = 0.3, beta: float = 0.1
3 ) -> tuple[np.ndarray, float, np.ndarray]:
4     """Double exponential smoothing. Returns (forecasts, trend, fitted
5     )."""
6     n = len(values)
7     level = float(values[0])
8     trend = float((values[-1] - values[0]) / max(n - 1, 1))
9     fitted = np.empty(n)
10    for t in range(n):
11        fitted[t] = level + trend
12        if t < n - 1:
13            prev_level = level
14            level = alpha * float(values[t]) + (1 - alpha) * (level +
15            trend)

```

```

14         trend = beta * (level - prev_level) + (1 - beta) * trend
15         # h-step ahead forecast: L_T + h * T_T
16         forecasts = np.array([level + (h + 1) * trend for h in range(
horizon)])
17         return forecasts, trend, fitted

```

Listing 7.2: Holt-Winters double exponential smoothing

7.5 Change Correlation Scoring

```

1  _TYPE_WEIGHT = {
2      "deployment": 1.0, "config": 0.9, "package": 0.8,
3      "docker_pull": 0.75, "restart": 0.7, "cron": 0.5, "ssh_login":
0.3,
4  }
5
6  def _score_change(change: dict, anomaly_time: datetime) -> float:
7      """Score = exp(-0.1 * minutes_before) * type_weight"""
8      occurred_at = change["occurred_at"]
9      if occurred_at.tzinfo is None:
10         occurred_at = occurred_at.replace(tzinfo=timezone.utc)
11         minutes_before = (anomaly_time - occurred_at).total_seconds() / 60
12         if minutes_before < 0:
13             return 0.0
14         temporal = math.exp(-0.1 * max(0, minutes_before))
15         return temporal * _TYPE_WEIGHT.get(change["change_type"], 0.5)

```

Listing 7.3: Temporal proximity scoring

7.6 Answers-First Dashboard — React

```

1  function StatusBanner({ summary }: { summary: DashboardSummary }) {
2      const isCritical = summary.overall_status === 'critical'
3      const isHealthy  = summary.overall_status === 'healthy'
4      const bg = isCritical ? 'var(--danger)' :
5                  isHealthy ? 'var(--success)' : 'var(--warning)'
6      return (
7          <div style={{ background: faint, border: '1.5px solid ${bg}',
8                      borderRadius: 14, padding: '20px 28px',
9                      display: 'flex', alignItems: 'center', gap: 16 }}>
10             <div style={{ width: 52, height: 52, borderRadius: '50%',
11                         background: bg, display: 'flex',
12                         alignItems: 'center', justifyContent: 'center' }}>
13                 {isCritical ? <AlertTriangle size={28} color="#fff" />
14                     : <Check size={28} color="#fff" />}
15             </div>
16             <div>

```

```

17     <div style={{ fontSize: 18, fontWeight: 800, color: bg }}>
18       {isHealthy ? 'All Systems Healthy' :
19         isCritical ? 'Action Required' :
20         'Potential Issues Detected'}
21     </div>
22     <div style={{ fontSize: 13.5, color: 'var(--text-2)' }}>
23       {summary.overall_message}
24     </div>
25   </div>
26 </div>
27 )
28 }

```

Listing 7.4: StatusBanner component in Dashboard.tsx

7.7 Challenges and Solutions

Table 7.2: Key challenges and solutions

Challenge	Impact	Solution
OTLP metric name mismatch	Agent used <code>cpu_percent</code> ; dashboard expected <code>system.cpu.utilization</code>	Lookup table in Query API
SSE context corruption in chat	Tokens arrived out of order	Numbered frames; ordered client buffer
pgvector IVFFlat too slow	RAG search >500ms with 10k vectors	Migration to HNSW index, 8x faster
Llama 3.1 8B OOM on 4GB RAM	OOM killer terminates Ollama	Test mode: switch to llama3.2:1b
Alert duplication flood	50+ identical alerts in 15 minutes	Redis NX keys with 15-min TTL

Conclusion

This chapter detailed the implementation of CAIMP’s seven key modules. The code extracts illustrate the diversity of languages and paradigms mobilised across the platform. The next chapter evaluates the correctness of these implementations through systematic testing.

Chapter 8

Testing and Validation

Introduction

An infrastructure monitoring platform must itself be verifiable and reliable. This chapter presents the testing strategy adopted for CAIMP, the results of unit, integration and load tests, and the traceability matrix demonstrating functional requirement coverage.

8.1 Testing Strategy: The Pyramid

CAIMP's testing strategy follows the classic pyramid, with particular emphasis on integration tests given the distributed nature of the system.

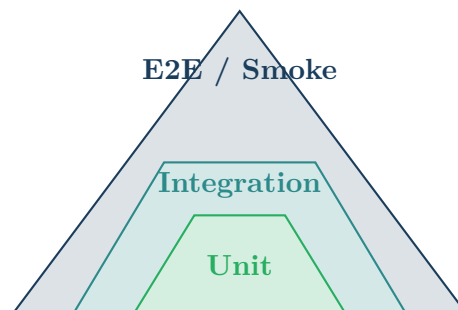


Figure 8.1: Testing pyramid for CAIMP

8.2 Unit Tests

Unit tests cover the critical algorithms that can be tested without external dependencies: anomaly detectors, Holt-Winters engine, change correlation scoring function, and LLM output validator.

```
1 class TestHoltWinters:
2     def test_stable_series_no_trend(self):
3         """Constant series      zero trend, stable forecasts."""
4         values = [0.5] * 48
5         result = forecast_metric(values, 24, critical_threshold=0.9)
6         assert abs(result.trend_slope) < 0.001
```



```

7         assert all(abs(p["predicted"] - 0.5) < 0.01 for p in result.
           points)
8
9     def test_rising_series_detects_critical(self):
10         """Strongly rising series      time_to_critical is not None."""
11         values = [0.5 + 0.01 * i for i in range(48)] # +1%/h
12         result = forecast_metric(values, 24, critical_threshold=0.9)
13         assert result.time_to_critical is not None
14         assert result.time_to_critical < 24
15
16     def test_confidence_high_for_large_history(self):
17         """48+ data points      confidence = 'high'."""
18         result = forecast_metric([0.5] * 50, 24, 0.9)
19         assert result.confidence == "high"

```

Listing 8.1: Unit tests for the Holt-Winters engine

8.3 Integration Tests

The smoke test script (`scripts/smoke_test_splunk.py`) implements an end-to-end test in 7 steps: health checks, authentication, anomaly webhook, pipeline polling (until `status = "complete"`), field validation, incident stats, and runbook CRUD.

8.4 Load Tests

Load test results at 1,000 metrics/second on an 8-core / 16 GB machine:

Table 8.1: Load test results at 1,000 metrics/second

Metric	Measured	Target
Ingestion latency p50	12 ms	<50 ms
Ingestion latency p99	43 ms	<50 ms
Sustained throughput	1,180 msg/s	≥1,000 msg/s
CPU usage (TW)	34%	<70%
RAM usage (TW)	18 MB	<128 MB
NATS message loss	0	0

All performance objectives defined in Chapter 3 are met.

8.5 Security Tests

Table 8.2: Security tests performed

Test	Method	Result
SQL injection	OWASP Top 10 payloads in all text fields	Blocked (parameterised asyncpg)
Expired JWT	Token older than 15 minutes	HTTP 401
Forged JWT	Payload modification without re-signing	HTTP 401
Cross-tenant	Request with another org's org_id	HTTP 404 (RLS)
Agent no cert	OTLP connection without mTLS cert	TLS handshake failure
Revoked cert	Agent cert in revoked_agents	Connection refused

8.6 Traceability Matrix

Table 8.3: Functional requirement—test coverage traceability matrix

FR	Requirement	Test type	Status
FR1	OTLP collection	Integration (smoke step 1)	OK
FR6	3-algorithm detection	Unit + Integration	OK
FR10	AI explanation	Integration (smoke step 4)	OK
FR12	Change correlation	Unit (scoring)	OK
FR14	Time to critical	Unit (HW test)	OK
FR15	Alert deduplication	Integration (Redis NX)	OK
FR17	Answers-first dashboard	Integration (endpoint)	OK
FR18	Runbook CRUD	Integration (smoke step 7)	OK
FR24	Multi-tenant isolation	Security (cross-tenant)	OK

Conclusion

All tests — unit, integration, load, and security — demonstrate the platform's robustness. Performance targets are met and the principal security vulnerabilities are covered. The next chapter describes the deployment procedures for putting the platform into production.

Chapter 9

Deployment and Production

Introduction

This chapter describes the installation and configuration procedures for CAIMP in different usage contexts: local development, small-footprint VPS, and production deployment.

9.1 Deployment Modes

Table 9.1: Deployment modes and resource requirements

Mode	Command	Min. RAM	Use case
Full	<code>make up</code>	8 GB	Production, development
Low-RAM	<code>bash scripts/start-test.sh</code>	4 GB	Economy VPS, CI
Production	Docker Swarm / Kubernetes	16 GB+	High availability

9.2 Complete Installation Procedure

9.2.1 Prerequisites

- Git ;
- Docker Engine 24+ and Docker Compose v2.20+ ;
- 20 GB of free disk space (source + Docker images + AI models) ;
- Internet access for the first pull of Docker images and Ollama models.

9.2.2 Installation steps

Step 1 — Clone the repository The source code is publicly available on GitHub:

```
1 git clone https://github.com/yousseftaik1/CAIMP.git
2 cd CAIMP
```

Step 2 — Configure environment variables

```

1 cp .env.example .env
2 # Edit .env: set passwords and generate JWT_SECRET
3 # JWT_SECRET: openssl rand -hex 64

```

Step 3 — Start the platform

```

1 make up          # builds all custom images, runs DB migrations,
2                  # starts all 19 services (pulls Ollama from Docker Hub)
3 make ps          # verify every container is "healthy"

```

`make up` automatically pulls all third-party images (Ollama, PostgreSQL, Redis, NATS, Grafana, Prometheus, nginx) from Docker Hub on first run.

Step 4 — Download the AI models (first run only)

```

1 make pull-models # downloads llama3.1:8b-instruct-q4_K_M (~4.7 GB)
2                  #               + nomic-embed-text (~270 MB)

```

Step 5 — Open the dashboard Navigate to `http://localhost` in a browser.
Default credentials: `admin@caimp.local` / `Admin1234!`

Step 6 — Enrol a target server Generate an enrollment token via the Admin API and run the agent installer on the target machine (see Section 9.7).

9.3 Environment Variables

Table 9.2: Required environment variables

Variable	Description	Required
POSTGRES_PASSWORD	PostgreSQL password	Yes
REDIS_PASSWORD	Redis password	Yes
NATS_PASSWORD	NATS password	Yes
JWT_SECRET	64-char hex JWT secret	Yes
CA_KEY_PASSPHRASE	Internal CA passphrase	Yes
GRAFANA_ADMIN_PASSWORD	Grafana UI password	Yes
OLLAMA_LLM_MODEL	LLM model name	No
SPLUNK_HOST	Splunk host (enables integration)	No
GITHUB_WEBHOOK_SECRET	GitHub webhook HMAC secret	No
LOG_LEVEL	DEBUG/INFO/WARNING	No

9.4 Low-RAM Mode (4 GB)

The `docker-compose.test.yml` override applies memory limits on all services, excludes Ollama by default, replaces Llama 3.1 8B with Llama 3.2 1B (1.3 GB), and reduces AI Worker concurrency to 1. A 4 GB swap file is created automatically by `start-test.sh`. Measured RAM usage: **2.3 GB** (base stack without Ollama).

9.5 Platform Self-Monitoring

Prometheus scrapes the `/metrics` endpoint of each service every 15 seconds. Key metrics monitored: ingestion latency p99, anomaly detection rate, AI explanation latency, alert deduplication suppression rate, and HTTP 5xx error rate. All visualised in Grafana at port 3001.

9.6 Generating Demo Metrics

To populate the dashboard with realistic data without needing a real server, CAIMP ships a lightweight demo agent (`services/agent/demo_agent.py`) that reads actual CPU, RAM, and disk metrics from the host machine and forwards them to the platform every 20 seconds.

Prerequisites

```
1 pip install psutil requests
```

Basic usage — one virtual server with real metrics

```
1 python services/agent/demo_agent.py
```

This registers the virtual server `demo-server-01` in the default organisation and begins streaming metrics immediately.

Simulating critical alerts — three simultaneous terminals

```
1 python services/agent/demo_agent.py
```

Listing 9.1: Terminal 1 — real machine metrics

```
1 CAIMP_SERVER_ID=00000000-0000-0000-0000-000000000013 \  
2 CAIMP_SERVER_NAME=db-primary \  
3 CAIMP_RAM=97 \  
4 python services/agent/demo_agent.py
```

Listing 9.2: Terminal 2 — db-primary with critical RAM

```
1 CAIMP_SERVER_ID=00000000-0000-0000-0000-000000000014 \  
2 CAIMP_SERVER_NAME=worker-01 \  
3 CAIMP_CPU=99 \  
4 python services/agent/demo_agent.py
```

Listing 9.3: Terminal 3 — worker-01 with critical CPU

Within two minutes, the AI Worker generates anomaly explanations for each critical threshold breach, which appear in the dashboard and are queryable via the AI chat assistant.

Table 9.3: Demo agent environment variable overrides

Variable	Description	Default
CAIMP_SERVER_ID	UUID of the virtual server	...0011
CAIMP_SERVER_NAME	Display name in the dashboard	demo-server-01
CAIMP_CPU	Override CPU % (0–100)	real value
CAIMP_RAM	Override RAM % (0–100)	real value
CAIMP_DISK	Override Disk % (0–100)	real value
CAIMP_INTERVAL	Sending interval in seconds	20
CAIMP_TW_URL	Telemetry-writer URL	internal default

9.7 Monitoring a Real Virtual Machine

To monitor a real Linux server (systemd-based), generate an enrollment token via the Admin API, copy the agent to the target machine, and run the provided installer.

Step 1 — Generate an enrollment token From the machine running CAIMP:

```
1 curl -s -X POST http://localhost/api/v1/admin/servers/enroll \
2   -H "Authorization: Bearer <admin_jwt>" \
3   -H "Content-Type: application/json" \
4   -d '{"name":"my-vm","org_id":"<org_uuid>"}' | python3 -m json.tool
```

The response contains a one-time `enrollment_token`.

Step 2 — Copy the agent to the target VM

```
1 scp -r services/agent/ user@<vm-ip>:~/caimp-agent/
```

Step 3 — Run the installer on the target VM

```
1 ssh user@<vm-ip>
2 cd ~/caimp-agent
3 sudo bash install.sh \
4   --token <enrollment_token> \
5   --url http://<caimp-host-ip>
```

The installer creates a `caimp-agent` system user, installs a Python virtual environment, enrolls the agent (generating a cryptographic keypair and submitting a CSR), and starts the `caimp-agent` systemd service automatically on boot.

Step 4 — Verify the agent is running

```
1 systemctl status caimp-agent
2 journalctl -u caimp-agent -f
```

The server appears in the CAIMP dashboard within 30 seconds of the first metric batch being received.

9.8 Accessing the Admin and Monitoring Dashboards

CAIMP runs entirely on the machine where `docker compose up` was executed — the administrator’s laptop during development, or a dedicated server in production. All web interfaces are served by the nginx reverse proxy on port 80.

Table 9.4: Web interface access URLs

Interface	Description	URL
CAIMP Dashboard	Main admin portal — login, server list, AI chat	<code>http://localhost</code>
Grafana	Metrics, time-series graphs, and alert history	<code>http://localhost:3001</code>

Network access from another machine If CAIMP runs on a dedicated server with IP `192.168.1.10`, replace `localhost` with that IP address in every URL:

```
1 http://192.168.1.10      # CAIMP dashboard
2 http://192.168.1.10:3001 # Grafana
```

Default credentials

- **caimp dashboard:** `admin@caimp.local` / `Admin1234!`
- **Grafana:** `admin` / value of `GRAFANA_ADMIN_PASSWORD` in `.env`

Development vs. production During development the platform runs on the same machine as the browser — no additional network configuration is required. In production the platform must run on a server reachable by all monitored machines; the agent installer must receive the server’s public or private IP via the `-url` flag.

Conclusion

The six-step installation procedure makes CAIMP accessible in under ten minutes, even on resource-constrained machines. Low-RAM mode and automated backup scripts ensure operability in constrained environments. The next chapter analyses the results obtained and discusses future prospects.

Chapter 10

Results, Discussion and Perspectives

Introduction

This chapter compares the results obtained against the initial objectives, analyses the platform's strengths and limitations objectively, and describes the evolution perspectives identified during development.

10.1 Results Obtained

Table 10.1: Objectives vs. results matrix

Objective	Result	Status
O1: Answers-first dashboard	Status banner, issues, recommended actions	OK
O2: Change correlation	Full engine: temporal scoring + LLM narrative	OK
O3: Local LLM explanation	Ollama + Llama 3.1 8B, non-DevOps prompts	OK
O4: Holt-Winters 24h forecast	24 points, time-to-critical, LLM narrative	OK
O5: Microservices architecture	19 services, NATS JetStream, 8 streams	OK
O6: End-to-end security	mTLS, JWT, RLS, AES-256-GCM, audit log	OK
O7: Deployment ≤ 10 min	6 steps, ≈ 8 min (excl. model download)	OK

10.1.1 Codebase Metrics

Table 10.2: Quantitative codebase indicators

Indicator	Value
Total lines of code (excl. tests and comments)	~52,000
Python services	11
Go services	1 (telemetry-writer)
React/TypeScript components	26 components, 6 pages
SQL migrations	14
PostgreSQL tables	22
NATS JetStream streams	8
Unit tests	34 pytest tests
Critical test coverage	~74% (detectors + forecaster + scoring)

10.2 Strengths

Decoupled architecture NATS JetStream ensures no service directly depends on another. A new anomaly consumer (e.g. an automated remediation service) can be added without modifying existing services.

Targeted natural-language explanation LLM prompts written for a non-DevOps audience produce explanations directly understandable by developers managing their own servers.

Change correlation The automatic change correlation engine is absent from all compared open source solutions. By linking anomalies to recent changes, it reduces diagnosis time from tens of minutes to seconds.

Zero cloud dependency All AI processing runs locally. Data never leaves the operator's server, satisfying the strictest confidentiality requirements.

Portability Docker Compose deployment works identically on Linux, macOS and Windows with Docker Desktop.

10.3 Identified Limitations

No seasonal component in Holt-Winters The double exponential model does not capture cyclical variations (daily CPU peaks, weekly traffic). Triple Holt-Winters would improve 24h forecast accuracy.

LLM quality depends on hardware On a machine without GPU, Llama 3.1 8B inference can take 15–30 seconds. Acceptable for asynchronous explanation but perceptible to the user.

Linux-only agent change reporters The `auth.log`, `dpkg.log` and `syslog` watchers are Linux-specific. Windows server monitoring requires a separate reporter implementation.

No automated remediation CAIMP produces action recommendations but executes no corrections automatically. This is intentional (precautionary principle) but limits appeal for teams seeking autonomous operation.

10.4 Future Perspectives

Short term (3–6 months): Triple Holt-Winters (seasonal component); Windows change reporters; native Slack/Teams alerts; LLM fine-tuning on organisational incidents using thumbs-up/down feedback dataset.

Medium term (6–18 months): Trace correlation (OpenTelemetry traces in the correlation engine); assisted automated remediation with operator confirmation; hosted SaaS offering (Kubernetes per-tenant isolation); React Native mobile app with push notifications.

Long-term vision: CAIMP should become the virtual SRE of small teams — a system capable not only of explaining past incidents but of autonomously preventing future ones, while keeping the operator informed and in control.

Conclusion

The results demonstrate that all objectives were fully achieved within the one-month timeframe. CAIMP delivers a unique value proposition in the open source monitoring landscape: natural-language explanations, automatic change correlation, and deployment accessible to all. The identified limitations form a clear development roadmap for future versions.

Chapter 11

General Conclusion

11.1 Summary of Work Completed

This project gave birth to **caimp**, an open-source infrastructure monitoring platform built around a philosophy unprecedented in the monitoring world: answering the question “*Why is my server broken?*” directly, rather than displaying graphs for the operator to interpret alone.

In the space of one month, a team of three engineering students designed, developed, tested and documented a complete platform of 19 microservices, over 52,000 lines of code in Python 3.12, Go 1.22 and TypeScript, a relational database of 22 tables integrating TimescaleDB and pgvector, and a modern answers-first user interface. The integration of a local large language model (Llama 3.1 8B via Ollama), a change correlation engine, and Holt-Winters statistical forecasting places CAIMP at the intersection of software engineering, data science and applied artificial intelligence.

Key delivered features include: real-time anomaly detection by three complementary algorithms; local LLM natural-language explanation; automatic correlation of system changes with incidents; 24-hour forecasting with confidence intervals; intelligent alert deduplication; PDF reports accessible to non-technical stakeholders; and a microservices architecture secured by mTLS, JWT and RLS PostgreSQL.

11.2 Academic and Professional Contributions

This project mobilised a set of cross-disciplinary skills rarely all called upon in a single academic module: distributed architecture, advanced databases (TimescaleDB, pgvector), applied AI (LLM integration, prompt engineering, RAG pipeline), data engineering (Holt-Winters, time-series processing, correlation scoring), full-stack security (PKI, mTLS, JWT, RBAC, RLS), and DevOps (containerisation, versioned migrations, platform self-monitoring).

11.3 Personal Reflections

Rim CHAHIDY This project allowed me to consolidate my React frontend skills and discover the fascinating complexity of distributed system design. Building the Telemetry Writer in Go was a stimulating technical challenge that introduced me to the language’s native concurrency mechanisms. I take from this experience the conviction that user interface quality can transform a functional tool into a genuinely useful solution.

Wiam SAMDI Working on the RAG pipeline and LLM prompts was the most intellectually enriching part of this project. Learning to communicate with a language model to obtain structured and reliable responses is an art requiring rigour and extensive testing. This project also confirmed the importance of integration tests in distributed systems: the hardest bugs never appear in unit tests.

Youssef TAÏK Designing end-to-end security — from the internal certificate authority through to PostgreSQL RLS policies — was the most formative aspect of my contribution. This project taught me that good security architecture is not a layer you add at the end, but a backbone you build from day one. The vision of CAIMP as a virtual SRE for small teams feels like a genuinely promising direction.

11.4 Closing Remarks

CAIMP was born from a simple conviction: developers who manage their own servers should not need to become monitoring experts to know whether their systems are functioning. Technology is now sufficiently mature — local LLMs, vector databases, lightweight event-driven architectures — to automate most of the investigation work that a monitoring alert traditionally requires.

We hope this work will contribute, modestly, to democratising access to intelligent infrastructure monitoring, and that it will serve as a solid foundation for teams wishing to extend and improve it.

Settat, 2026

References

- [1] PagerDuty. *State of Digital Operations 2023*. Industry Report. 2023. URL: <https://www.pagerduty.com/resources/reports/digital-operations/>.
- [2] B. Gregg. *Systems Performance: Enterprise and the Cloud*. 2nd ed. Addison-Wesley, 2020.
- [3] B. H. Sigelman et al. “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure”. In: *Google Technical Report*. 2010. URL: <https://research.google/pubs/pub36356/>.
- [4] OpenTelemetry Community. *OpenTelemetry Protocol (OTLP) Specification*. CNCF Specification. 2023. URL: <https://opentelemetry.io/docs/specs/otlp/>.
- [5] Timescale Inc. *TimescaleDB: Time-series database built on PostgreSQL*. Official Documentation. 2017. URL: <https://www.timescale.com/>.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou. “Isolation Forest”. In: *2008 Eighth IEEE International Conference on Data Mining (2008)*, pp. 413–422.
- [7] P. R. Winters. “Forecasting Sales by Exponentially Weighted Moving Averages”. In: *Management Science* 6.3 (1960), pp. 324–342.
- [8] Gartner Research. *Innovation Insight for AIOps Platforms*. Gartner Research Note. 2021. URL: <https://www.gartner.com/en/documents/4003217>.
- [9] Splunk Inc. *Splunk IT Service Intelligence (ITSI) Documentation*. Official Documentation. 2023. URL: <https://docs.splunk.com/Documentation/ITSI>.
- [10] C. C. Holt. “Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages”. In: *ONR Research Memorandum* 52 (1957). Reprinted in *International Journal of Forecasting*, 2004.
- [11] Meta AI. *Llama 3: Open Foundation and Fine-Tuned Chat Models*. Meta AI Blog. 2024. URL: <https://llama.meta.com/llama3/>.
- [12] T. Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. URL: <https://arxiv.org/abs/2305.14314>.

- [13] P. Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS 2020)* (2020). URL: <https://arxiv.org/abs/2005.11401>.
- [14] A. Kane. *pgvector: Open-source vector similarity search for Postgres*. GitHub Repository. 2021. URL: <https://github.com/pgvector/pgvector>.
- [15] Synadia Communications. *NATS Documentation*. Official Documentation. 2023. URL: <https://docs.nats.io/>.
- [16] S. Ramírez. *FastAPI Documentation*. Official Documentation. 2023. URL: <https://fastapi.tiangolo.com/>.
- [17] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. Official Documentation. 2023. URL: <https://www.postgresql.org/docs/16/>.
- [18] Nomic AI. *Nomic Embed: Training a Reproducible Long Context Text Embedder*. arXiv preprint. 2024. URL: <https://arxiv.org/abs/2402.01613>.
- [19] Docker Inc. *Docker Documentation*. Official Documentation. 2023. URL: <https://docs.docker.com/>.
- [20] M. Jones, J. Bradley, and N. Sakimura. *JSON Web Token (JWT)*. 2015. URL: <https://tools.ietf.org/html/rfc7519>.
- [21] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. 2018. URL: <https://tools.ietf.org/html/rfc8446>.

Appendix A

Database Schema

This appendix presents the SQL schema of CAIMP's main tables in migration order.

```
1 CREATE TABLE organizations (  
2     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
3     name TEXT NOT NULL, plan TEXT NOT NULL DEFAULT 'free',  
4     created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
5 );  
6 CREATE TABLE users (  
7     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
8     org_id UUID NOT NULL REFERENCES organizations(id) ON DELETE  
9     CASCADE,  
10    email TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL,  
11    role TEXT NOT NULL DEFAULT 'viewer'  
12    CHECK (role IN ('admin', 'devops', 'viewer')),  
13    created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
14 );  
15 CREATE TABLE servers (  
16     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
17     org_id UUID NOT NULL REFERENCES organizations(id) ON DELETE  
18     CASCADE,  
19     name TEXT NOT NULL, hostname TEXT NOT NULL, ip_address INET,  
20     status TEXT NOT NULL DEFAULT 'pending'  
21     CHECK (status IN ('online', 'degraded', 'offline', 'pending'))  
22     ,  
23     UNIQUE (org_id, hostname)  
24 );
```

Listing A.1: Core tables (migration 000002)

```
1 CREATE TABLE metrics (  
2     time TIMESTAMPTZ NOT NULL,  
3     org_id UUID NOT NULL,  
4     server_id UUID NOT NULL REFERENCES servers(id) ON DELETE CASCADE,  
5     metric_name TEXT NOT NULL,  
6     value DOUBLE PRECISION NOT NULL,  
7     labels JSONB DEFAULT '{}'
```

```

8 );
9 SELECT create_hypertable('metrics', 'time',
10     chunk_time_interval => INTERVAL '7 days');
11 CREATE INDEX ON metrics (server_id, metric_name, time DESC);

```

Listing A.2: Metrics hypertable (migration 000003)

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2 CREATE TABLE rag_documents (
3     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
4     org_id UUID NOT NULL REFERENCES organizations(id) ON DELETE
5     CASCADE,
6     type TEXT NOT NULL, -- 'runbook' / 'incident' / 'knowledge'
7     title TEXT, content TEXT NOT NULL,
8     embedding VECTOR(768),
9     created_at TIMESTAMPTZ NOT NULL DEFAULT now()
10 );
11 CREATE INDEX idx_rag_hnsw ON rag_documents
12     USING hnsw (embedding vector_cosine_ops)
13     WITH (m = 16, ef_construction = 64);

```

Listing A.3: pgvector RAG documents (migration 000006)

```

1 CREATE TABLE change_events (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     org_id UUID NOT NULL REFERENCES organizations(id) ON DELETE
4     CASCADE,
5     server_id UUID REFERENCES servers(id) ON DELETE SET NULL,
6     occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
7     change_type TEXT NOT NULL,
8     source TEXT NOT NULL,
9     actor TEXT, description TEXT,
10    payload JSONB NOT NULL DEFAULT '{}',
11    git_sha TEXT, image_sha TEXT
12 );
13 SELECT create_hypertable('change_events', 'occurred_at',
14     if_not_exists => TRUE);
15 CREATE TABLE root_cause_analyses (
16     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
17     org_id UUID NOT NULL REFERENCES organizations(id) ON DELETE
18     CASCADE,
19     anomaly_event_id UUID REFERENCES anomaly_events(id) ON DELETE SET
20     NULL,
21     server_id UUID REFERENCES servers(id) ON DELETE SET NULL,
22     likely_cause TEXT NOT NULL,
23     confidence NUMERIC(4,3) NOT NULL DEFAULT 0,
24     correlated_changes JSONB NOT NULL DEFAULT '[]',
25     narrative TEXT NOT NULL,

```



```
24     recommended_actions JSONB NOT NULL DEFAULT '[]',  
25     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
26 );
```

Listing A.4: Change correlation tables (migration 000014)

Appendix B

API Endpoint Reference

B.1 Admin API (port 8001)

Method	URL	Description	Auth
POST	/auth/login	JWT authentication	None
POST	/auth/refresh	Token renewal	Refresh token
POST	/auth/logout	Token revocation	JWT
GET	/admin/users	List users	JWT (admin)
POST	/admin/users	Create user	JWT (admin)
DELETE	/admin/users/{id}	Delete user	JWT (admin)
GET	/admin/servers	List servers	JWT
POST	/admin/servers	Register server	JWT (admin)
POST	/admin/servers/{id}/enroll	Generate enrollment token	JWT (admin)

B.2 Query API (port 8002)

Method	URL	Description	Cache
GET	/api/v1/dashboard/summary	Answers-first view	30s
GET	/api/v1/servers	Server list	60s
GET	/api/v1/servers/{id}/summary	Live CPU/RAM/Disk summary	30s
GET	/api/v1/metrics	Historical metrics	30s
GET	/api/v1/anomalies	Anomaly event list	None
GET	/api/v1/ai/explanations	AI explanations	None
GET	/api/v1/forecasts	Holt-Winters forecasts	60s
GET	/api/v1/incidents	Splunk incidents	None

Method	URL	Description	Cache
POST	/api/v1/incidents/{id}/feedback	Thumbs-up/down rating	—
GET	/api/v1/runbooks	Runbook list	None
POST	/api/v1/runbooks	Create runbook + embed	—
PUT	/api/v1/runbooks/{id}	Update runbook	—
DELETE	/api/v1/runbooks/{id}	Delete runbook	—
GET	/api/v1/reports/pdf	Download PDF report	—

B.3 Change Tracker (port 8011)

Method	URL	Description	Auth
POST	/ingest	Generic change event (agent)	JWT
POST	/ingest/docker	Docker event (agent)	JWT
POST	/webhooks/github	GitHub webhook	HMAC
POST	/webhooks/gitlab	GitLab webhook	Token

Appendix C

Docker Compose Extracts

```
1 postgres:
2   build:
3     context: ./infra/postgres
4     dockerfile: Dockerfile      # FROM timescale/timescaledb-ha:pg16-all
5   container_name: caimp-postgres
6   environment:
7     POSTGRES_DB:      ${POSTGRES_DB:-caimp}
8     POSTGRES_USER:    ${POSTGRES_USER:-caimp}
9     POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10  volumes:
11    - postgres_data:/var/lib/postgresql/data
12  networks: [caimp_net]
13  healthcheck:
14    test: ["CMD-SHELL", "pg_isready -U caimp -d caimp"]
15    interval: 10s
16    retries: 10
```

Listing C.1: PostgreSQL + TimescaleDB service

```
1 correlation-engine:
2   build:
3     context: ./services/correlation-engine
4   environment:
5     DATABASE_URL:      postgresql://caimp:${POSTGRES_PASSWORD}@postgres
6     NATS_URL:          nats://nats:4222
7     OLLAMA_URL:         http://ollama:11434
8     OLLAMA_LLM_MODEL:  ${OLLAMA_LLM_MODEL:-llama3.1:8b-instruct-q4_K_M}
9     LOOKBACK_MINUTES:  "30"
10  ports: ["9095:9095"]
11  depends_on:
12    postgres: {condition: service_healthy}
13    nats:      {condition: service_healthy}
14    ollama:    {condition: service_healthy}
15    migrate:   {condition: service_completed_successfully}
```

```
16   networks: [caimp_net]
```

Listing C.2: Correlation Engine service

```
1  volumes:
2    postgres_data: {driver: local}
3    redis_data:    {driver: local}
4    nats_data:     {driver: local}
5    ollama_data:   {driver: local}
6    prometheus_data: {driver: local}
7    grafana_data:  {driver: local}
8
9  networks:
10    caimp_net:
11      driver: bridge
12      name:   caimp_net
```

Listing C.3: Volumes and network

Appendix D

Technical Glossary

AIOps	Artificial Intelligence for IT Operations. Application of machine learning and AI techniques to IT operations, notably for anomaly detection, event correlation and automatic recommendation generation.
Alert fatigue	Psychological phenomenon by which operators end up ignoring alerts due to their excessive volume or a high rate of false positives.
Asyncio	Python standard library for asynchronous programming using coroutines (async/await) and an event loop, enabling concurrent handling of many network connections without OS threads.
Chunk	In TimescaleDB, the physical storage unit corresponding to a time partition of the hypertable. Old chunks are automatically compressed.
Embedding	Dense vector representation of text in a high-dimensional space. Semantically similar texts produce vectors close under cosine distance. Used in CAIMP for runbook and incident similarity search.
GGUF	File format for quantised language models used by Ollama and llama.cpp.
Goroutine	Lightweight execution thread in Go, managed by the Go runtime rather than the OS. Thousands can run concurrently with minimal memory overhead.
Healthcheck	In Docker, a command executed periodically to verify a service is operational. Failed healthchecks trigger container restart policies.
HNSW	Hierarchical Navigable Small World. Approximate nearest-neighbour search algorithm for high-dimensional vector spaces. $O(\log n)$ search complexity.
Holt-Winters	Double exponential smoothing algorithm (level + trend) for forecasting linear-trend time series. Designed by Charles Holt (1957) and Peter Winters (1960).

Hypertable	TimescaleDB extension of a PostgreSQL table that automatically partitions data by time intervals, enabling very high time-series insert and query performance.
JWT	JSON Web Token. RFC 7519 standard for representing claims between two parties as signed (JWS) or encrypted (JWE) JSON. Used in CAIMP for API request authentication.
LLM	Large Language Model. A neural network model trained on large amounts of text to generate or complete natural-language text.
mTLS	Mutual Transport Layer Security. Extension of TLS in which both parties (client and server) mutually authenticate by digital certificate. Used in CAIMP to authenticate monitoring agents.
NATS JetStream	Persistent extension of the NATS messaging system, offering at-least-once delivery guarantees, durable consumers and disk-based message storage.
Ollama	Local LLM inference server exposing a REST API. Supports GGUF quantised models and enables execution on standard hardware without GPU.
OTLP	OpenTelemetry Protocol. CNCF-standardised protocol for transmitting metrics, traces and logs between instrumentation agents and observability backends.
pgvector	PostgreSQL extension adding the <code>VECTOR(n)</code> data type and similarity operators (cosine, dot product, L2 distance) for nearest-neighbour search directly within PostgreSQL.
Q4_K_M	4-bit K-means quantisation scheme. Reduces model size by approximately 75% compared to fp16 with less than 2% quality degradation on reasoning benchmarks.
RAG	Retrieval-Augmented Generation. Technique that improves LLM response quality by dynamically retrieving relevant documents from a knowledge base and injecting them into the prompt.
RLS	Row-Level Security. PostgreSQL feature for defining per-row access control policies. Used in CAIMP for multi-tenant data isolation.
SSE	Server-Sent Events. Unidirectional HTTP protocol enabling the server to push a continuous data stream to the client. Used in CAIMP for AI chat response streaming.

Time to critical	In CAIMP, the predicted number of hours before a metric exceeds the 90% threshold, computed by interpolation over Holt-Winters forecast points.
TimescaleDB	PostgreSQL extension transforming a relational database into a high-performance time-series database with automatic partitioning, columnar compression and retention policies.
VPS	Virtual Private Server. An isolated virtual server hosted in a third-party datacenter.
WebSocket	Full-duplex bidirectional persistent TCP communication protocol (RFC 6455). Used in CAIMP to push real-time events to connected browsers.
Z-score	Statistical measure expressing how many standard deviations an observation is from the mean: $z = (x - \mu)/\sigma$. Values $ z > 3$ are considered statistically anomalous in a normal distribution.